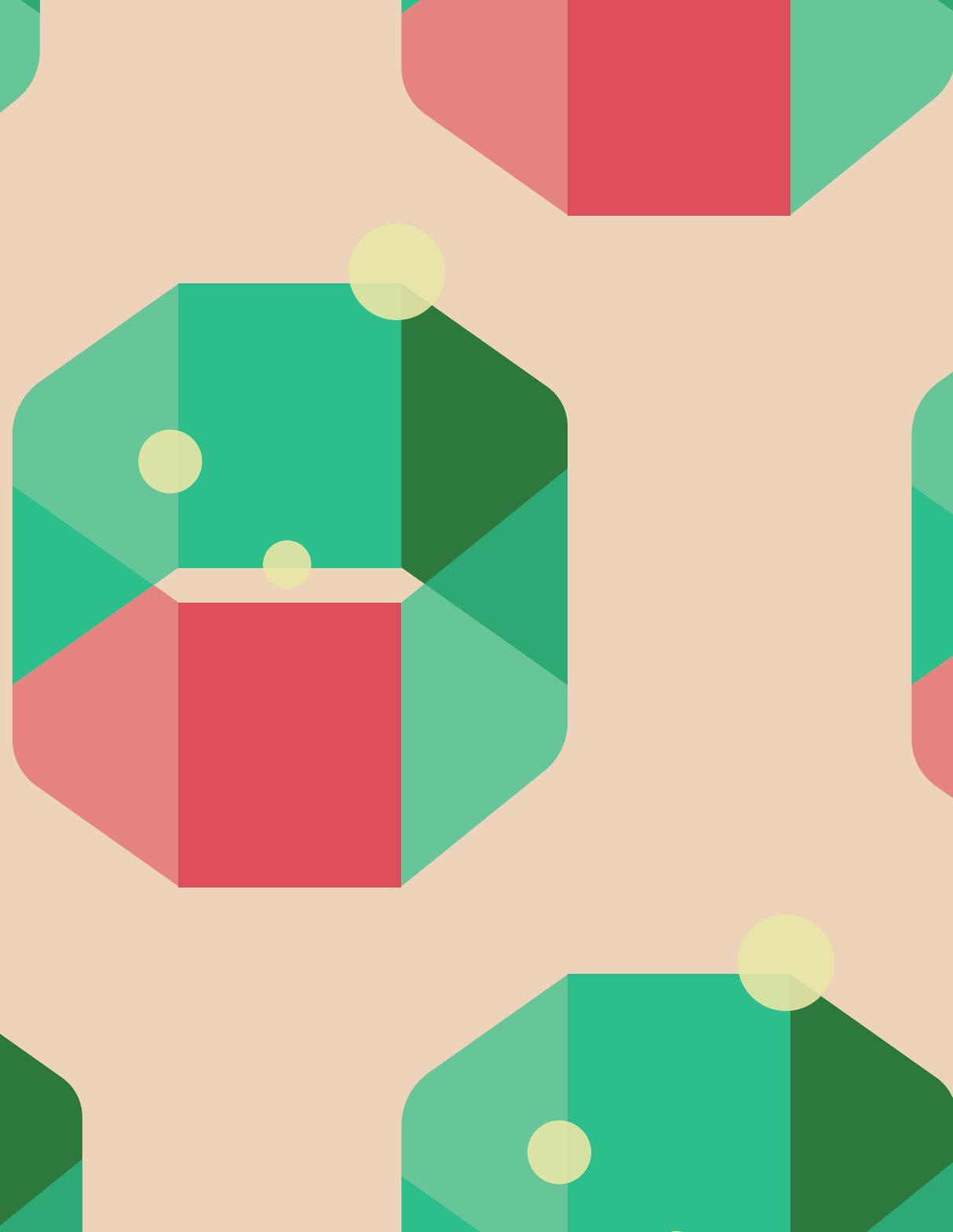




Backend Development

2nd half of
course.

- Server = * computer

- should run 24/7

- application = * logic that runs on server

- * responds to request.

- * 404 → request is invalid.

- database = * store user data

node.js → we will learn in
the course.

professionally most
popular.

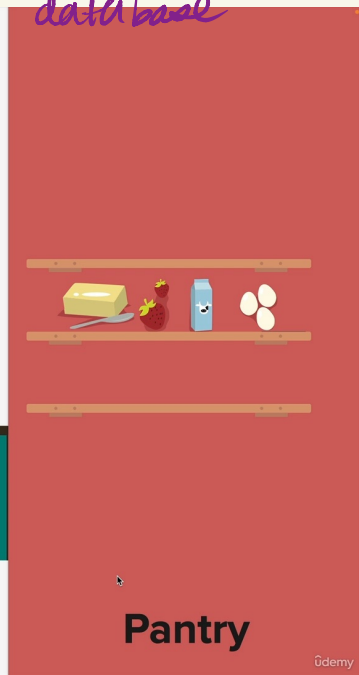
node.js

Javascript runtime framework

frontend

back end

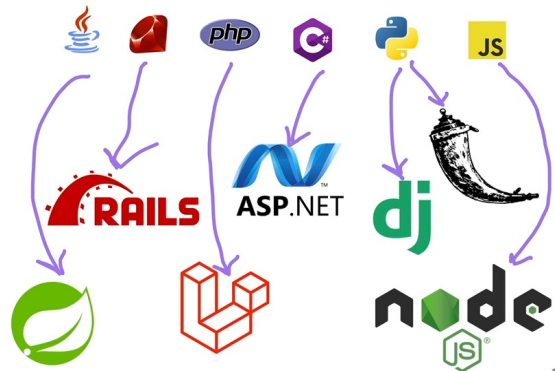
database



Front-end



Back-end



All different technologies for frontend and back end.

* web framework provides builtin functionalities.
saves time, energy.

* node.js → JavaScript runtime.

V8 Engine → powers the chrome browser

Asynchronous → • does not have to do everything sequentially.

Node **R E P L**
read eval print loop

initiate by giving node command

> To run a js file using node:

node <filename>

↳ runs the program

```

5 index.js > fs.writeFile("message.txt", " Hello my name is Rimjhim and I am learning Node js") callback
1  const fs = require("fs");
2  fs.writeFile("message.txt", " Hello my name is Rimjhim and I am learning Node js", (err) => {
3    if (err) throw err;
4    console.log("The file has been saved and u dont need to worry");
5  })

```

readFile → will return a buffer if no encoding is specified

NPM

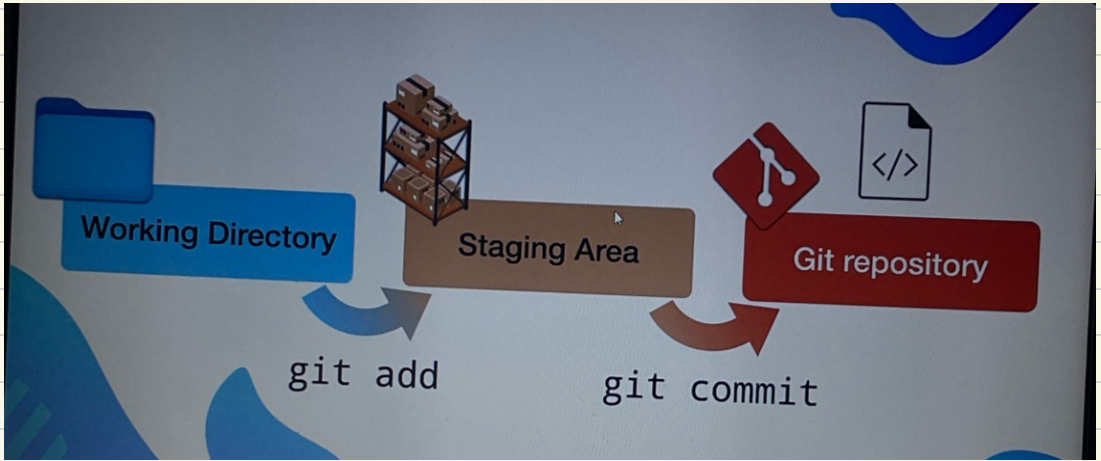
- managed by github
- holds all node stuff people have made

node package

manager.

- it is a community of functionalities
- open source
- you can search packages.

Version control using git.



you can control and keep track of the different versions of your program.

* use `git check out` to return to the last version.

" `git diff filename` "

↳ look at the differences in your program accordingly.

* git checkout filename

↳ this will roll back to the last version that was committed to the repository.

upload local repo to github

1. `git remote add origin <url>`
2. `git push -u origin main.`

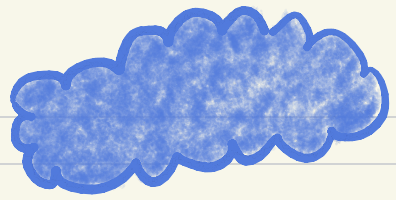
Note: never push secret keys and API keys to your github, leads to security breach

↪ use .gitignore

don't add OS originated files. (local computer stuff)

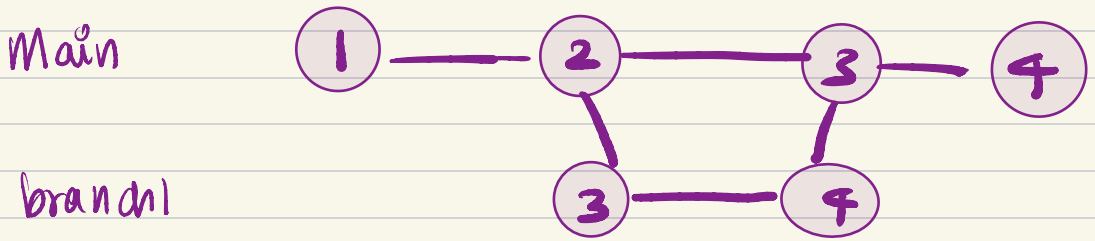
1. `touch .gitignore` (create file)
2. add the filenames that you want to ignore.
3. you can also just add specific extensions

that you want to ignore



Branching

we want to try to do something different.
make a new branch to do your experiments.



work simultaneously on the main branch and other branches and then merge when you're done with your experiments

* `git branch branch-name`
↳ create a branch

* `git checkout branch-name`
↳ enter that branch

* `git branch` → tells you the branches.

* `git merge branch-name`

→ you should be on the main branch

→ when you're satisfied with the other branch,
do `merge branch-name`

→ a vim editor opens and you can add a
merge message.

→ now the changes from `branch-name`
are absorbed by the main branch

Forking

good angela : owner

Evil angela : read-only

making a duplicate of a remote repo and adding it under your account.

- * When you fork a repository then you own it,
- * you can change anything you want,

Pull request.

- * you have only read access, and you have cloned the repo and made changes.
- * now you want the owner of the repo to incorporate those changes into the main repo.
- * you make a pull request.
- * the owner decides if they want to pull the changes that the read-only person made
- * then, approve the changes and

merge pull request.

Useful commands

`ls -a` → to see all the hidden files.

`git rm --cached -r .` →

remove whatever you have added to the staging area using `add`.